

宝鍵

誰が見てもわかりやすい設計の **実践** と、その効果の **実感**

東京情報デザイン専門職大学
情報デザイン学部 情報デザイン学科 3年
岸田 匠馬

ジャンル : ヴァンサバライク・アクション

ゲーム概要



宝鍵 (ヴァンサバライク・アクション)

敵を倒して無色鍵を集め、宝箱をすべて開けるゲーム

- ・宝箱からはその色に応じたスキル・ウルトを持つ色鍵がゲットできる
- ・すべての宝箱を開けばクリア、HPが0になればゲームオーバー
- ・HPと無色鍵は**共通リソース**
→宝箱を開けるタイミングが重要

開発環境・スペック

開発人数	5人
開発状況	ゲームジャム期間中(2日間) 開発へ参加 その後はチームメンバーへ引き継ぎ
使用ツール	Unity/C#, git/github, VSCode
プラットフォーム	Windows, Web (Unityroom)
担当範囲	・Chestシステムの設計・実装 ・Keyシステムの設計・実装 ・Skill/Ultの基盤システムの設計・実装 ・BigSwordUlt, TimeStopUltの実装 ・プロジェクト全体のマネジメント

工夫点① Interfaceによる拡張性の高いシステム設計

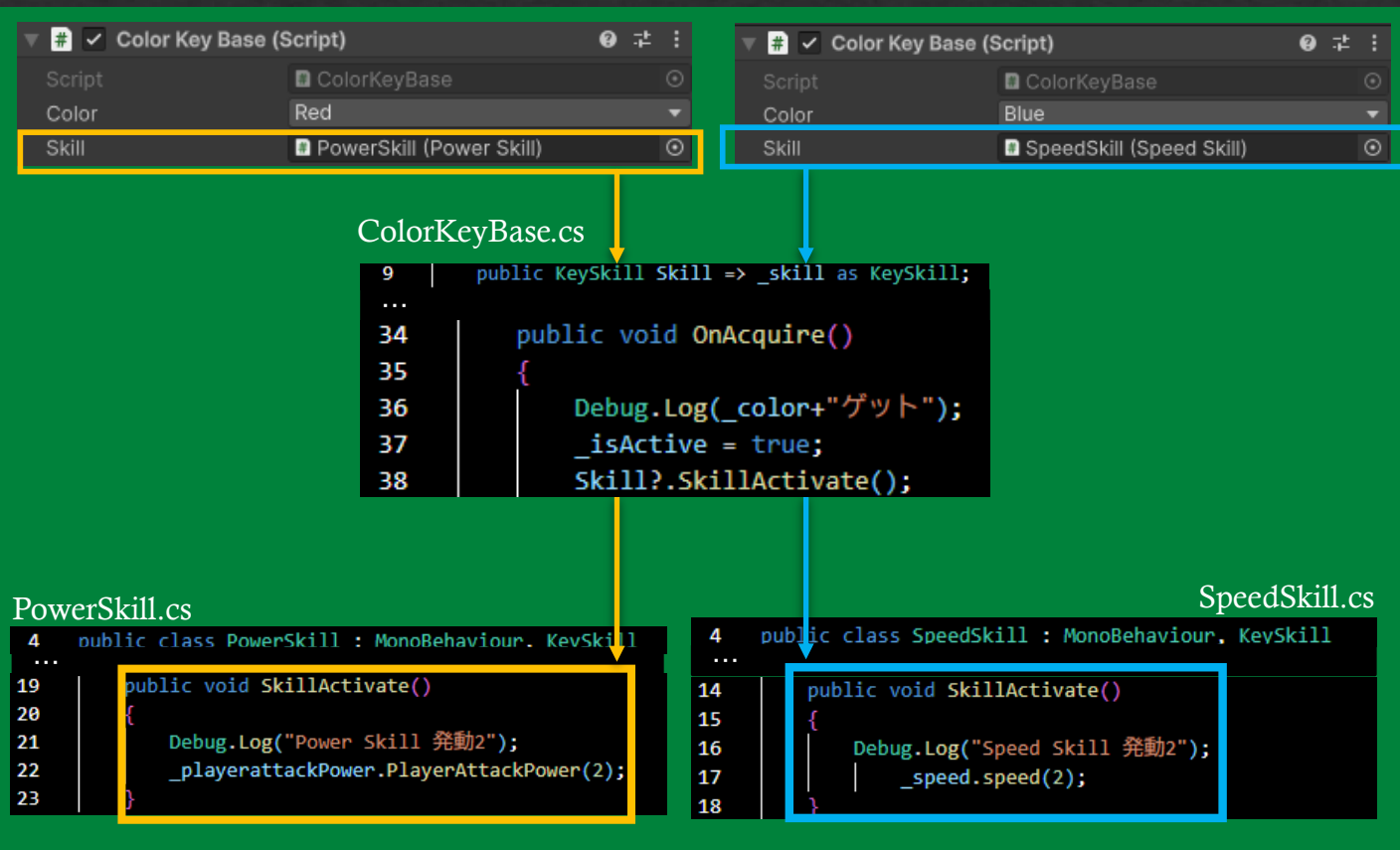
効果① 能力が増えてもコードが複雑化しづらい

Before (以前の作品): 条件分岐と処理が集中

```
277 else if ((*this->magic)[2] == "自爆") {
278     animsOther.push_back(std::make_pair(AnimationClip(), center));
279     animsOther.back().first.SetInterval(0.1);
280     animsOther.back().first.SetTextures(selffxps);
281     animsOther.back().first.SetIsLoop(false);
282     jibakuSE.playOneShot();
283 }
284 if ((*this->magic)[2] == "ゴリラ") {
285     gorillaSE.playOneShot();
286     if (type == 3) {
287         animsOther.push_back(std::make_pair(AnimationClip(), center));
288         animsOther.back().first.SetInterval(0.1);
289         animsOther.back().first.SetTextures(gorilla);
290     }
291     else {
292         for (int i = 0; i < dir.size(); i++) {
293             animsOther.push_back(std::make_pair(AnimationClip(), GorillaPosCulc(i)));
294             animsOther.back().first.SetInterval(0.1);
295             animsOther.back().first.SetTextures(gorilla);
296         }
297     }
298 }
299 else if ((*this->magic)[2] == "たぬき") {
300     tanukiSE.playOneShot();
301     for (int i = 0; i < dir.size(); i++) {
302         GenTanukiRute(dir[i]);
303         animsOther.push_back(std::make_pair(AnimationClip(), otherEffectRute.back().first));
304         animsOther.back().first.SetInterval(0.1);
305         animsOther.back().first.SetTextures(tanuki);
306     }
307 }
308 else if ((*this->magic)[2] == "バナナ") {
309     bananaSE.playOneShot();
310     for (int i = 0; i < dir.size(); i++) {
311         GenBananaGoal(dir[i]);
312         animsOther.push_back(std::make_pair(AnimationClip(), otherEffectRute.back().first));
313         animsOther.back().first.SetTextures(banana);
314         effectStopFlg = true;
315     }
316 }
317 else if ((*this->magic)[2] == "スライム") {
318     for (int i = 0; i < dir.size(); i++) {
319         animsOther.push_back(std::make_pair(AnimationClip(), SlimePos(dir[i]));
320         animsOther.back().first.SetInterval(0.1);
321         animsOther.back().first.SetTextures(slime[type]);
322         effectStopFlg = true;
323     }
324     slimeSE.playOneShot();
325 }
```

プロのコードからInterfaceを学んだ

After: Interfaceを用いて分離



分岐をSkillクラスへ分離したことで、ColorKeyBaseに処理を追加する必要がなくなり、さらに 1つ1つのコードが短くなるため、複雑化しづらくなった。

工夫点① Interfaceによる拡張性の高いシステム設計

効果② 比較的簡単に能力を追加できる

追加手順

1. ソールファイル新規作成
2. Interface実装
3. 新たな能力の処理を追加
4. 鍵に割り当てる

以上

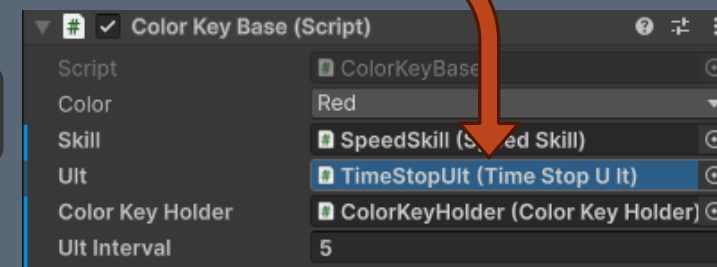
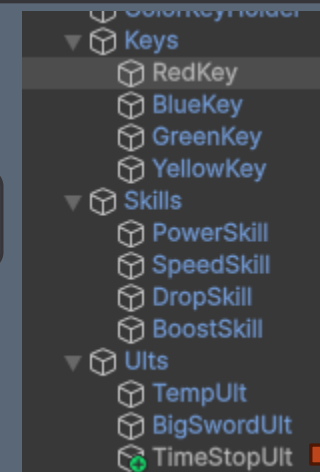
1. 新規作成

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4
5 0 個の参照
6 public class TempUlt : MonoBehaviour, KeyUlt
7 {
8     2 個の参照
9     public void UltActivate()
10     {
11         Debug.Log("Temp Ult 発動");
12         //ここに処理を追加
13     }
14 }
15
```

2. Interface実装

3. 処理追加

4. 鍵に割り当てる



動画でも紹介しています : <https://youtu.be/8uimVm92ymo>

実際にUnityの使用経験が少ないメンバーでも新規スキルを2種類(PowerSkillとSpeedSkill)追加でき、設計による開発効率向上を確認できた。

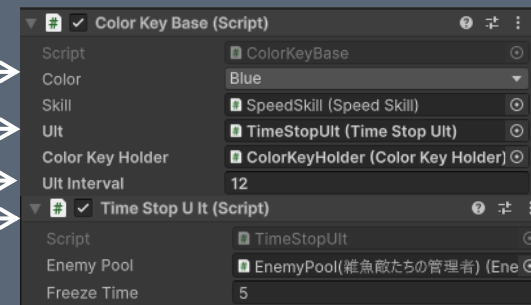
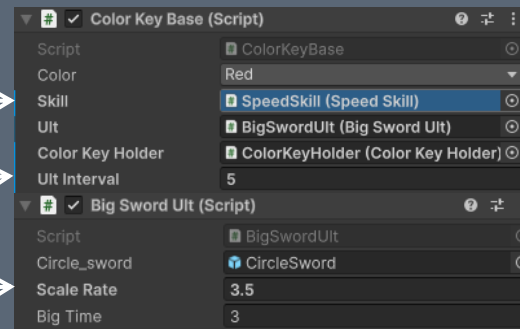
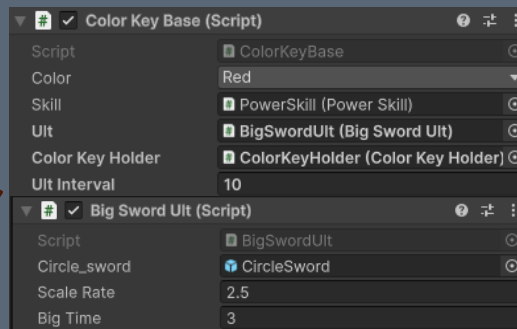
工夫点② Unity Editor上で完結するレベルデザイン環境

以前、プランナーの人から、プログラムを触らなくてもパラメータ調整できる環境がよいとアドバイスをされたため、今回はレベルデザインにかかわるパラメータはInspectorで表示されるよう、シリアライズにして実装した。

シリアライズすることで
Inspectorから調整可能に

```
3 public class ColorKeyBase : MonoBehaviour
4 {
5     2 個の参照 | @ Unity シリアル化フィールド (複数の値)
6     [SerializeField] private KeyColor _color;
7     1 個の参照 | @ Unity シリアル化フィールド (複数の値)
8     [SerializeField] private MonoBehaviour _skill;
9     1 個の参照 | @ Unity シリアル化フィールド (複数の値)
10    [SerializeField] private MonoBehaviour _ult;
11    1 個の参照 | @ Unity シリアル化フィールド = ColorKeyHolder
12    [SerializeField] private ColorKeyHolder colorKeyHolder;
13    2 個の参照
14    public KeySkill Skill => _skill as KeySkill;
15    1 個の参照
16    public KeyUlt Ult => _ult as KeyUlt;
17
18    1 個の参照 | @ Unity シリアル化フィールド (複数の値)
19    [SerializeField] private float _ultInterval = 5f;
```

```
5 public class BigSwordUlt : MonoBehaviour, KeyUlt
6 {
7     1 個の参照 | @ Unity シリアル化フィールド = CircleSword
8     [SerializeField] private GameObject _circle_sword;
9     2 個の参照 | @ Unity シリアル化フィールド = 2.5
10    [SerializeField] private float _scaleRate = 2;
11    1 個の参照 | @ Unity シリアル化フィールド = 3
12    [SerializeField] private float _bigTime = 3;
```



UnityEditor上のみでスキル・ウルトや色などの組み合わせを即座に試すことができるため、
レベルデザインやデバックに非常に効率的に取り組むことが可能になった。

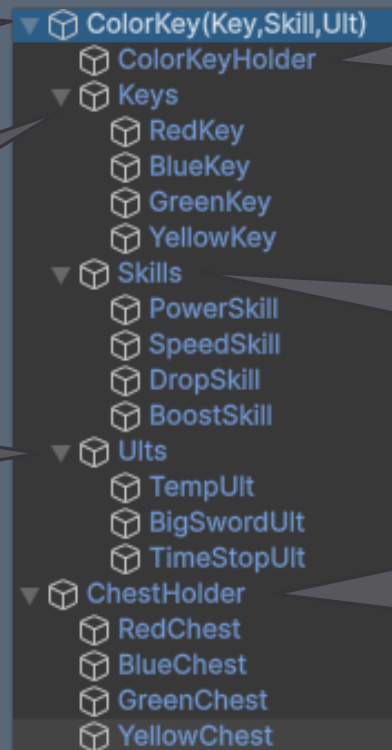
工夫点② Unity Editor上で完結するレベルデザイン環境

また、Unity Editorをメインに触ることになるため、それぞれのオブジェクトがどこにあるかがわかりやすいように、Hierarchyにコンテナを配置するようにした。

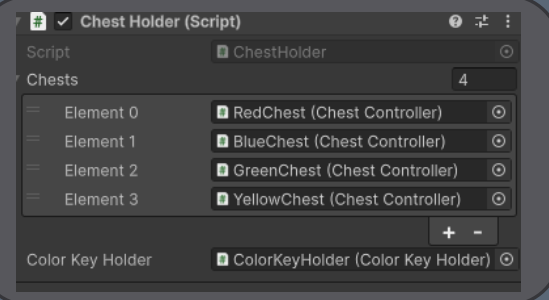
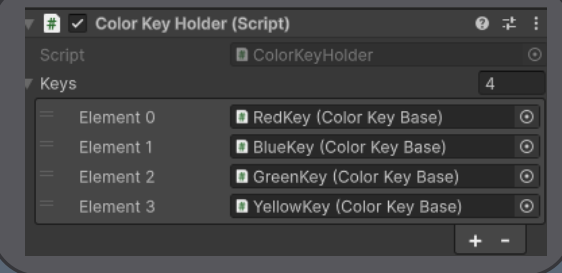
鍵に関するすべてのオブジェクトが入ったコンテナ

鍵オブジェクト(ColorKeyBase)が入ったコンテナ

ウルトオブジェクト(KeyUlt Interfaceがついたクラス)が入ったコンテナ



スキルオブジェクト(KeySkill Interfaceがついたクラス)が入ったコンテナ



この経験の価値・今後の展望

オブジェクト指向による設計の考え方とそのメリットを実感した。

- ◇ 今回の作品を通して、「だれが見てもわかる設計にする」とは、基盤となるシステムを作ることによって、**経験の少ない人でもそれを利用して実装できる環境**を用意することだと実感した。
- ◇ 過去の作品で学んだ、
 1. 継承やInterfaceなどのプログラム手法の重要性
 2. 状況に応じた継承・Interfaceの選択
 3. 共通処理の度合いによる使い分けという教訓をフルに生かし、Interfaceを用いて実装した結果、経験の少ない人でも機能を追加できたことで、作業効率が大幅に向上し、Interfaceの実装が成功したと感じられた。
- ◇ 次の機会では、今回十分に活用できなかった「継承」についての理解を深め、継承とInterfaceの違いについて知り、適切な使い分けをできるようになる。